

Inteligența artificială ca instrument pentru dezvoltatorii de software

Stadiul actual, instrumente, impact și perspective

Gratii Stanislav
Specializarea: SIC
Suceava, 2026

1. Introducere

1.1 Contextul general

Dezvoltarea software a suferit transformări profunde de-a lungul anilor. De la programarea în cod mașină și limbaje de asamblare, până la limbaje de nivel înalt, cadre moderne de lucru și metodologii agile, industria IT s-a adaptat și reinventat continuu. Totuși, niciuna dintre aceste schimbări nu a fost la fel de rapidă și disruptivă precum integrarea instrumentelor bazate pe inteligența artificială în procesul de scriere a codului.

Începând cu 2021, odată cu lansarea GitHub Copilot și accesul larg la modelele lingvistice mari (LLM - Large Language Models), inteligența artificială a trecut de la un domeniu exclusiv academic la un instrument uzual pentru programatori. Astăzi, conform studiilor realizate de GitHub, Stack Overflow și McKinsey, peste 70% dintre dezvoltatorii profesioniști folosesc cel puțin un astfel de instrument AI în activitatea lor zilnică.

1.2 Scopul si obiectivele raportului

Acest raport își propune sa ofere o imagine completa si riguroasa asupra următoarelor aspecte:

- Principalele categorii de instrumente AI utilizate de programatori
- Impactul cuantificabil asupra productivității si calității codului
- Avantajele demonstrate prin studii
- Riscurile si limitările identificate in literatura de specialitate
- Perspectivele de viitor

1.3 Metodologia

Raportul se bazează pe analiza literaturii academice publicate in reviste indexate ISI si IEEE, pe studii de piața realizate de companii de referința (GitHub, JetBrains, Stack Overflow, McKinsey, Gartner), pe documentația tehnica a principalelor instrumente AI, si pe date statistice publice privind adoptarea si impactul acestor tehnologii. Perioada acoperita este 2020-2025. Si toate reprezentările grafice incluse în lucrare au fost realizate cu suportul unor instrumente bazate pe inteligență artificială, utilizând date extrase din sursele bibliografice menționate.

2. Fundamentele Inteligentei Artificiale in Dezvoltarea Software

2.1 Modele lingvistice mari (LLM)

Majoritatea instrumentelor AI moderne destinate programatorilor se bazează pe arhitecturi de tip transformer și pe modele lingvistice mari (LLM) [15]. Aceste modele sunt pre-antrenate pe cantități uriașe de date textuale și de cod sursă, dobândind capacitatea de a înțelege contextul, de a completa secvențe de cod, de a detecta erori și de a genera explicații în limbaj natural [3], [13].

Cele mai influente modele de bază care alimentează instrumentele actuale includ: GPT-4 și GPT-4o (OpenAI), Claude 3 Opus și Sonnet (Anthropic), Gemini 1.5 Pro (Google DeepMind), CodeLlama și Llama 3 (Meta), și DeepSeek Coder (DeepSeek AI) [14]. Fiecare dintre aceste modele a fost antrenat pe sute de miliarde de tokeni, incluzând cod sursă în zeci de limbaje de programare [3], [4].

2.2 Tipuri de sarcini gestionate de IA

Instrumentele AI pentru programatori pot asista la o gamă largă de sarcini. Fără a epuiza toate posibilitățile, domeniul evoluează rapid, iar noi categorii de utilizare continuă să apară, mai jos sunt prezentate cinci categorii principale care acoperă aria funcțională esențială și care au fost, totodată, valorificate în mod direct în cadrul aplicației dezvoltate în acest proiect:

Categorie	Exemple de sarcini	Instrumente reprezentative
Generare de cod	Autocompletare, generare de functii,	GitHub Copilot, Cursor, Tabnine
Revizuire si debugging	Detectare bug-uri, sugestii de refactorizare	CodeRabbit, DeepCode, SonarQube AI
Documentare	Generare comentarii, README, documentatie API	Mintlify, Swimm, Copilot
Testare	Generare unit tests, teste de integrare	CodiumAI, Diffblue, Copilot
Securitate	Detectare vulnerabilitati, audit cod	Snyk, GitHub Advanced Security, CodeQL AI

3. Impactul asupra Productivității Dezvoltatorilor

3.1 Date privind productivitatea dezvoltatorilor

Cuantificarea impactului inteligenței artificiale asupra productivității programatorilor a fost analizată în mai multe studii riguroase. Cel mai cunoscut este un experiment controlat realizat de GitHub, publicat în 2023 în seria NBER Working Paper. În acest studiu, 95 de programatori au fost împărțiți aleatoriu în două grupuri: unul care a utilizat GitHub Copilot și unul care nu a folosit acest instrument. Rezultatele au demonstrat că grupul care a folosit Copilot a finalizat sarcinile cu **55,8% mai rapid** decât grupul de control [1], [22].

Un al doilea studiu major, realizat de cercetători de la Harvard Business School în 2023, a examinat impactul instrumentelor AI asupra consultanților profesioniști care efectuau sarcini complexe de cunoaștere. Participanții care au folosit AI au completat sarcinile cu 37% mai rapid și au obținut scoruri de calitate cu 18% mai bune. Studiul a semnalat însă și efectul de „plafonare”: pentru sarcinile de dificultate ridicată, îmbunătățirile erau semnificativ mai mici sau inexistente [2].

McKinsey Global Institute (2023) estimează că instrumentele AI pot accelera activitățile de dezvoltare software cu 20–45%, în funcție de complexitatea sarcinii și de experiența programatorului. Sarcinile care beneficiază cel mai mult includ: scrierea de cod boilerplate (+70%), generarea de teste unitare (+63%) și reautorizarea codului existent (+55%) [5].

Sursă / An	Metodologie	Metric principal	Rezultat cheie	Limitare semnalată
GitHub / NBER (2023)	Experiment controlat randomizat; 95 programatori împărțiți în 2 grupuri	Viteza de finalizare a sarcinii	+55,8% mai rapid față de grupul de control	Sarcini limitate la un singur tip de task (HTTP server)
MIT (2023)	Consultanți profesioniști; sarcini de scriere și cod; grup ChatGPT vs. control	Viteză (+37%) și calitate (+18%)	+37% viteză / +18% calitate output	Efect de 'plafonare' la sarcini de dificultate ridicată
McKinsey Digital (2023)	Analiză de piață; date agregate din multiple companii și roluri	Accelerare activități de dezvoltare software	+20–45% general; +70% cod boilerplate	Variabilitate mare în funcție de complexitate și experiență

Tabel sintetic – studii empirice privind impactul AI asupra productivității

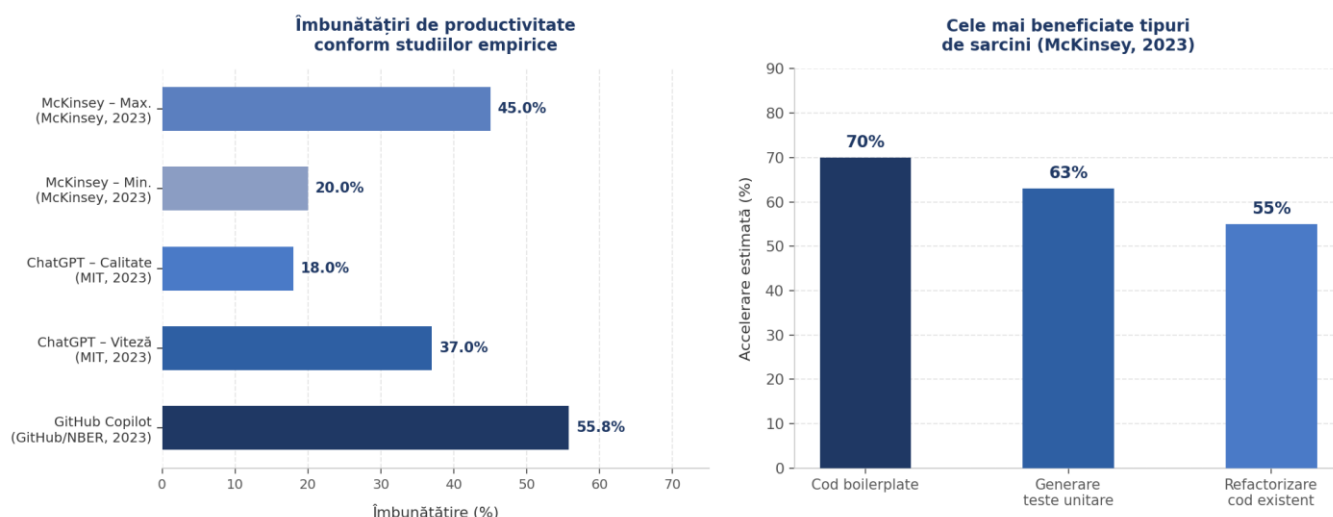


Figura 3.1 – Comparație îmbunătățiri de productivitate raportate (stânga: per studiu; dreapta: per tip de sarcină, McKinsey 2023)

Diferența semnificativă între rezultatele studiului GitHub (55,8%) și cele ale MIT (37%) sugerează că impactul AI depinde puternic de tipul de sarcină. Studiul NBER a utilizat un task bine delimitat și repetitiv (implementarea unui server HTTP), propice autocompletării ceea ce explică câștigul mai mare. Sarcinile mai complexe sau mai creative, cum sunt cele din studiul MIT, produc câștiguri mai moderate, dar totuși substanțiale.

3.2 Impactul diferențiat în funcție de experiență

Un aspect critic care reiese din literatura recentă de specialitate este că impactul AI asupra productivității nu este uniform el variază semnificativ în funcție de nivelul de experiență al programatorului. Cele mai riguroase date provin dintr-un set de trei experimente de teren randomizate (RCT) realizate de cercetători de la MIT, Microsoft, Princeton și Wharton, publicate în 2024 sub titlul „The Effects of Generative AI on High Skilled Work”. Studiul a inclus peste 4.800 de programatori din Microsoft, Accenture și o corporație Fortune 100 anonimă, măsurând săptămânal numărul de pull request-uri, commit-uri și build-uri finalizate [16].

Tabel comparativ – impactul AI pe nivel de experiență

Studiu / Sursă	An	Metodologie	Juniori / Începători	Seniori / Experimentați	Obs. cheie
Cui, Demirer et al. (MIT / Microsoft / Wharton)	2024	RCT; 4.867 dev.; 3 companii (Microsoft, Accenture, Fortune 100)	+27–39% productivitate	+8–13% productivitate	Efect de egalizare: juniorii adoptă și beneficiază mai mult
Paradis et al. (Google)	2024	Evaluare internă: autocomplete, smart paste, NL-to-code	Câștiguri mai mari la cei mai puțin experimentați	+21% reducere timp/task	Seniorii verifică mai riguros → câștig mai mic dar mai sigur
Gambacorta, Qiu, Shan & Rees (BIS / Ant Group)	2024	CodeFuse (agent AI); metric: linii de cod; angajați Alibaba	Câștiguri mai mari la juniori; 20% din +55% total = AI direct	Seniori: efect nesemnificativ statistic	Efecte AI autentice, nu doar volum – calitate verificată
Uplevel / GitClear (industrie)	2024	Analiză date din producție; commit-uri AI vs. non-AI	+41% bugs față de non-AI	Seniorii resping sau revizuiesc mai des codul AI	Viteza câștigată poate fi anulată de debugging suplimentar

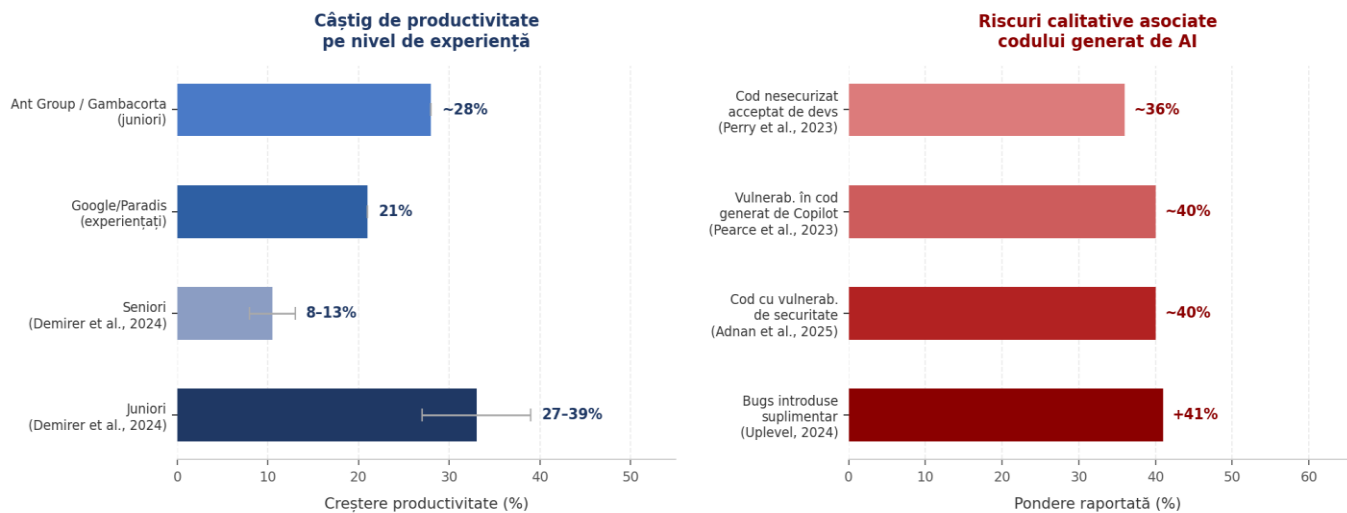


Figura 3.2 – Stânga: câștig de productivitate pe nivel de experiență (Demirer et al., 2024; Paradis et al., 2024; Gambacorta et al., 2024). Dreapta: riscuri de calitate asociate codului generat de AI (Uplevel, 2024; Adnan et al., 2025; Pearce et al., 2023; Perry et al., 2023).

Câștigul de productivitate al juniorilor vine cu un risc calitativ documentat. Studiul Uplevel (2024) a arătat că programatorii care utilizează Copilot introduc cu 41% mai multe bug-uri față de cei care nu îl folosesc [19]. Perry et al. (2023) au observat că participanții care au folosit asistenți AI au livrat mai mult cod nesecurizat, iar paradoxal, credeau că au scris cod mai sigur decât în realitate un fenomen de supra încredere (overtrust) [20]. Adnan et al. (2025) estimează că 30–50% din eșantioanele de cod AI analizate conțin vulnerabilități critice de securitate [21]. Aceste cifre subliniază că pentru programatorii juniori, câștigul de viteză trebuie contrabalansat de practici sistematice de code review și de conștientizarea limitelor instrumentelor AI.

Un efect secundar, mai puțin discutat, este impactul AI asupra procesului de onboarding. Conform datelor longitudinale Faros AI (2024–2025), timpul până la al 10-lea pull request indicator standard de integrare în echipă s-a redus la jumătate în organizațiile cu utilizare AI [19]. Aceasta sugerează că instrumentele AI acționează ca un multiplicator de integrare rapidă, nu doar de productivitate individuală.

Pe termen mediu, reducerea decalajului de competențe între juniori și seniori pentru sarcinile de rutină poate restructura dinamica echipelor de dezvoltare: seniorii vor fi din ce în ce mai valoroși pentru supervizare, arhitectură și validarea critică a outputului AI, în timp ce juniorii vor putea contribui mai rapid la sarcini de implementare [16], [17], [18].

3.3 Satisfacția profesională si reducerea sarcinilor repetitive

Dincolo de productivitatea pură, instrumentele AI contribuie semnificativ la satisfacția profesională a programatorilor prin eliminarea sarcinilor percepute ca plictisitoare sau repetitive. Datele disponibile provin din mai multe surse independente și converg spre un tablou consistent: adoptarea AI îmbunătățește starea de bine subiectivă a dezvoltatorilor, deși cu anumite nuanțe și limitări relevante.

Studiul realizat de GitHub Research în 2022, bazat pe sondajul a peste 2.000 de programatori care utilizau GitHub Copilot [22], a produs una dintre cele mai complete radiografii ale impactului psihologic al AI în programare. Conform acestuia, 87% dintre respondenți au raportat că AI îi ajută să conserve efortul mental la sarcinile repetitive, iar 73% că rămân mai ușor în starea de flux (flow state) o condiție de imersiune cognitivă asociată cu satisfacția maximă în muncă. Între 60 și 75% dintre utilizatori au raportat că se simt mai împliniți profesional, mai puțin frustrați la codare și că pot lucra la probleme mai satisfăcătoare.

La nivelul comunității mai largi de programatori, Stack Overflow Developer Survey 2024 [23] cu peste 65.000 de respondenți din 185 de țări arată că 70% dintre profesioniști nu percep AI ca o amenințare la locul de muncă. Sentimentul favorabil față de instrumentele AI s-a menținut la 70%+ în 2023 și 2024, deși a scăzut la 60% în 2025, conform ediției ulterioare a aceluiași sondaj [24]. Această tendință de

scădere poate reflecta o maturizare a așteptărilor, pe măsură ce utilizatorii devin mai conștienți de limitele reale ale instrumentelor.

O revizuire sistematică a 37 de studii empirice publicată la AMCIS 2024 [25] confirmă că automatizarea sarcinilor triviale și repetitive este unul dintre cele mai frecvent raportate beneficii ale asistenților AI (alături de accelerarea dezvoltării și reducerea căutărilor de cod). Dimensiunile Satisfacție, Performanță și Eficiență din cadrul SPACE Framework sunt cel mai intens investigate, indicând că satisfacția profesională este recunoscută ca un vector esențial al productivității nu un simplu efect secundar.

Tabel sintetic – satisfacție și percepție AI în rândul programatorilor

Sursă / Ref.	An	Eșantion	Rezultat principal	Observație
GitHub Research [22]	2022	2.000+ programatori Copilot	87% – conservare efort mental; 73% – flow state; 60–75% – satisfacție crescută	Sondaj de percepție; participanții sunt utilizatori activi Copilot (nu grup de control)
Stack Overflow Dev. Survey [23]	2024	65.000+ dev., 185 țări	70% nu percep AI ca amenințare; sentiment favorabil >70%	Sondaj reprezentativ; acoperire globală, toate nivelurile de experiență
Stack Overflow Dev. Survey [24]	2025	49.000+ dev.	Sentiment favorabil scade la 60%; 46% nu au încredere în acuratețea AI	Tendință de maturizare / scădere a entuziasmului față de 2023–2024
AMCIS 2024 – SLR [25]	2024	37 studii empirice 2014–2024	Automatizare sarcini repetitive – beneficiu raportat în majoritate studiilor	Cognitive offloading și reducere colaborare în echipă – riscuri identificate
GitHub Octoverse 2024 [26]	2024	Milioane dev. pe GitHub, oct. 2023 – sep. 2024	+59% contribuții proiecte gen. AI; Python depășește JavaScript ca limbaj #1	Creștere explozivă a implicării cu AI; indicator indirect al adopției și entuziasmului

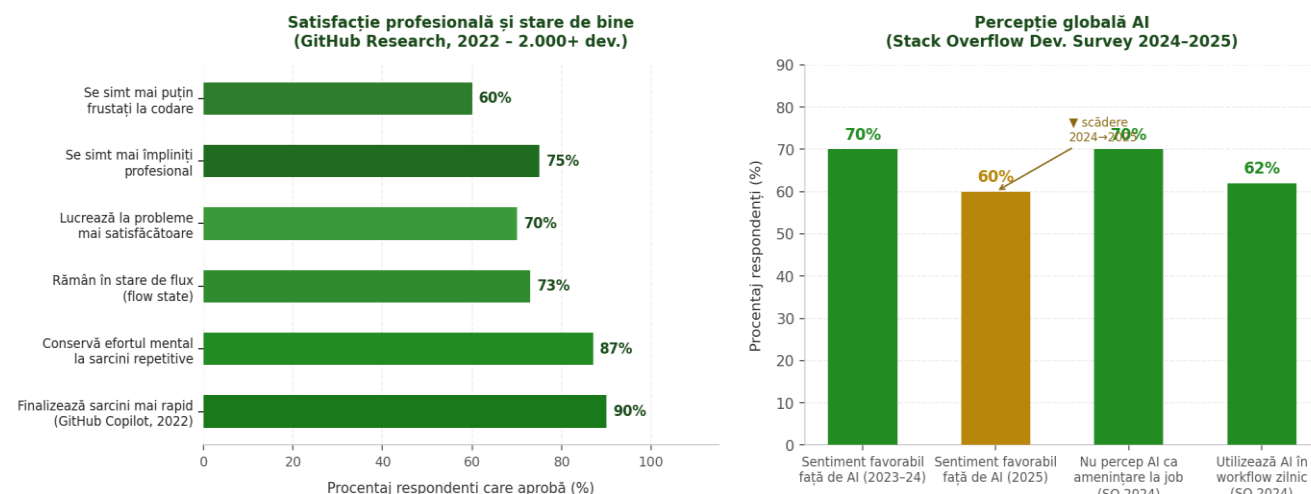


Figura 3.3 – Stânga: indicatori de satisfacție și stare de bine raportați de utilizatorii. Dreapta: evoluția percepției globale AI în rândul programatorilor.

Scăderea sentimentului favorabil față de AI de la 70%+ (2023–2024) la 60% în 2025 [24], combinată cu creșterea neîncrederii în acuratețea outputului (46% activ neîncrezători în 2025 față de 31% în 2024), indică un fenomen de dezamăgire post-entuziasm. Acest tipar este consistent cu ciclul Gartner Hype Cycle: după vârful de așteptări exagerate urmează o perioadă de dezamăgire înainte de stabilizare la un nivel matur de adopție. Pentru organizații, acesta este semnalul că simpla disponibilizare a instrumentelor AI nu este suficientă — sunt necesare formare, ghiduri de utilizare și practici de code review pentru a susține satisfacția pe termen lung.

5. Avantaje Demonstrate

5.1 Accelerarea ciclului de dezvoltare

Unul dintre cele mai clare avantaje ale integrării AI în fluxul de dezvoltare este comprimarea ciclului de la idee la produs funcțional. Instrumentele moderne permit prototiparea rapidă: un programator poate descrie în limbaj natural funcționalitatea dorită și obține un prototip funcțional în minute, nu ore. Startup-urile și echipele mici beneficiază în mod special de această capacitate, putând concura cu echipe mult mai mari.

5.2 Accesibilizarea programării

Unul dintre cele mai semnificative efecte ale instrumentelor AI este democratizarea accesului la programare. Platformele low-code și no-code alimentate de AI, precum și asistenții de tip chat (ChatGPT, Claude, Gemini), permit astăzi persoanelor fără

background tehnic solid să creeze aplicații funcționale un lucru practic imposibil în urmă cu doar câțiva ani [3], [14].

Această accesibilizare este reală și măsurabilă: un antreprenor fără cunoștințe de programare poate construi un prototip funcțional, un cercetător își poate automatiza procesarea datelor, iar un student poate livra un proiect web fără să fi studiat vreodată HTML sau JavaScript [4], [13]. Instrumentele AI acționează, în acest context, ca un „coleg virtual” disponibil 24/7, care traduce intenția în cod [1], [22].

Însă această democratizare vine cu un avertisment important: aplicațiile create de utilizatori fără experiență tehnică tind să fie funcționale la suprafață, dar fragile în profunzime. Codul generat poate conține bug-uri subtile, vulnerabilități de securitate și arhitecturi care nu scalează [6], [20]. Utilizatorul neexperimentat nu are, de regulă, instrumentele necesare pentru a identifica aceste probleme și, mai grav, nici nu știe că ele există [19], [21].

Cu alte cuvinte, AI a coborât dramatic pragul de intrare în programare, dar nu a eliminat importanța competenței tehnice. Oricine poate construi astăzi o aplicație; nu oricine poate construi una sigură, scalabilă și ușor de întreținut. Această distincție devine din ce în ce mai importantă pe măsură ce aplicațiile create cu asistență AI ajung în producție și gestionează date reale ale utilizatorilor [5], [8].

5.4 Detectarea proactivă a erorilor

Erorile de securitate descoperite în producție costă în medie de 6 ori mai mult decât cele identificate în faza de dezvoltare, iar cele descoperite după un incident de securitate real pot costa de sute de ori mai mult, atât financiar cât și reputațional [5], [8]. În acest context, detectarea proactivă a erorilor reprezintă unul dintre cele mai clare cazuri de utilizare cu return on investment demonstrabil pentru instrumentele AI în dezvoltarea software.

Instrumentele AI de analiză statică de nouă generație precum DeepCode, Snyk Code sau GitHub Advanced Security pot identifica clase întregi de vulnerabilități (SQL injection, XSS, buffer overflow) înainte ca codul să ajungă în producție [6], [26]. Spre deosebire de analizatoarele statice tradiționale bazate pe reguli, modelele AI pot detecta vulnerabilități bazate pe context, nu doar pe pattern-uri predefinite ceea ce reduce semnificativ atât rata de fals-pozitive, cât și pe cea de fals-negative [6], [20].

Un avantaj important este integrarea acestor instrumente direct în fluxul de lucru al programatorului. Feedback-ul de securitate nu mai vine după zile sau săptămâni, printr-un raport extern de audit, ci în timp real, în editorul de cod, în momentul în care vulnerabilitatea este introdusă [22], [26]. Acest model de „shift-left security” mutarea verificărilor de securitate cât mai devreme în ciclul de dezvoltare este considerat de industrie drept una dintre cele mai eficiente strategii de reducere a riscului [5], [13].

Există însă o tensiune fundamentală de semnalat: aceleași instrumente AI care ajută la detectarea vulnerabilităților pot, în același timp, să genereze cod vulnerabil atunci când sunt folosite ca asistenți de programare [6], [19], [21]. Studiile arată că programatorii care folosesc asistenți AI au tendința de a accepta sugestii de cod fără a le verifica din perspectiva securității, bazându-se implicit pe presupunerea că AI-ul a generat cod sigur, o presupunere adesea incorectă [20]. Prin urmare, detectarea proactivă a erorilor prin AI nu elimină necesitatea culturii de securitate în echipă, ci o completează.

6. Riscuri si Limitări

6.2 Dependenta si diminuarea competentelor

Există îngrijorări legitime în comunitatea academică și industrială că dependența excesivă de instrumentele AI ar putea diminua abilitățile fundamentale de programare, în special la nivelul generației de programatori care începe să învețe direct cu asistența AI. Analogia cu calculatoarele de buzunar și aritmetica mentală este frecvent invocată în aceste dezbateri [7], [10].

Un studiu publicat în IEEE Transactions on Software Engineering (2024) a urmărit o cohortă de 200 de studenți care au învățat să programeze folosind GitHub Copilot intensiv, comparativ cu un grup de control. După 18 luni, grupul cu AI a demonstrat o înțelegere mai slabă a algoritmilor fundamentali și a structurilor de date, deși a produs cod funcțional mai rapid în sarcinile curente [7], [13].

7. Tendințe si Direcții de Viitor

7.1 Agenți autonomi și integrarea în ciclul de viață al software-ului

Următoarea frontieră în AI pentru programatori sunt agenții autonomi: sisteme capabile să preia o sarcină complexă descrisă în limbaj natural și să o ducă la îndeplinire complet autonom scrierea codului, crearea testelor, configurarea infrastructurii și trimiterea unui pull request pentru revizuire. Proiecte precum Devin (Cognition AI), SWE-Agent (Princeton) și OpenDevin au demonstrat că agenții AI pot rezolva autonom până la 14% din problemele reale raportate pe GitHub Issues un progres semnificativ față de 0% în urmă cu trei ani [11], [14]. Deși procentajele rămân modeste, traiectoria este clară.

În paralel, AI începe să depășească faza de implementare și să se extindă în întreg ciclul de viață al software-ului: analiza cerințelor, arhitectura sistemului, testarea formală și operațiunile (AIOps). Instrumente experimentale bazate pe Lean 4 sau Coq încep să fie utilizate pentru a demonstra matematic că anumite clase de erori nu pot apărea într-un program o direcție promițătoare pentru domeniile critice [8], [5]. Această extindere promite câștiguri de productivitate semnificative, dar ridică și întrebări despre distribuția responsabilității atunci când un agent autonom introduce erori în producție [2], [9].

8. Studii de Caz

Pentru realizarea designului interfeței, a fost utilizată platforma Stitch un instrument AI generativ specializat în crearea de prototipuri web, care interpretează specificații în limbaj natural și generează cod HTML/CSS/JavaScript funcțional. Alegerea acestei platforme a fost motivată de capacitatea sa de a produce rapid prototipuri vizuale fără scrierea manuală a întregului cod de interfață.

Prompt-ul utilizat a fost:

„Create a modern, clean, and highly responsive Single Page Application (SPA) web design for a university event management platform. The application should feature a professional color scheme based on shades of blue, a clear navigation structure, an events listing section with cards, a registration/login area, and a hero section with a call-to-action button. All UI components should follow modern design standards with smooth hover effects, clear typography, and a well-organized layout suitable for university students and staff.”

Prompt-ul a fost formulat cu intenție, incluzând specificații precise referitoare la tipul aplicației, domeniul funcțional, paleta cromatică, componentele UI solicitate și publicul țintă.

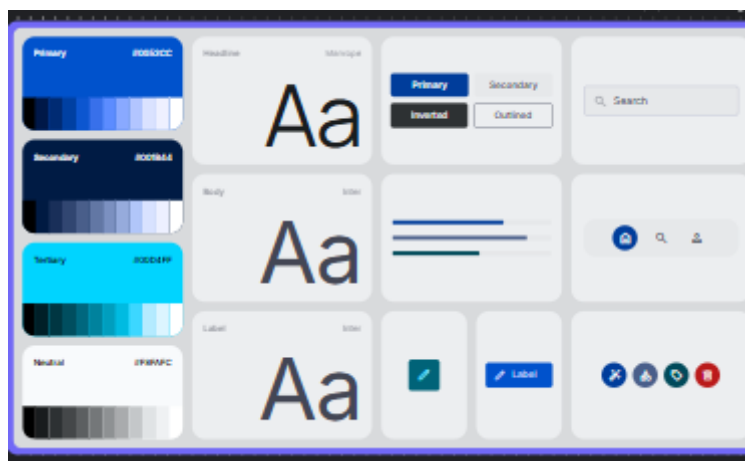


Figura 1. Designul interfeței SPA generat de AI prin platforma Stitch (Google)

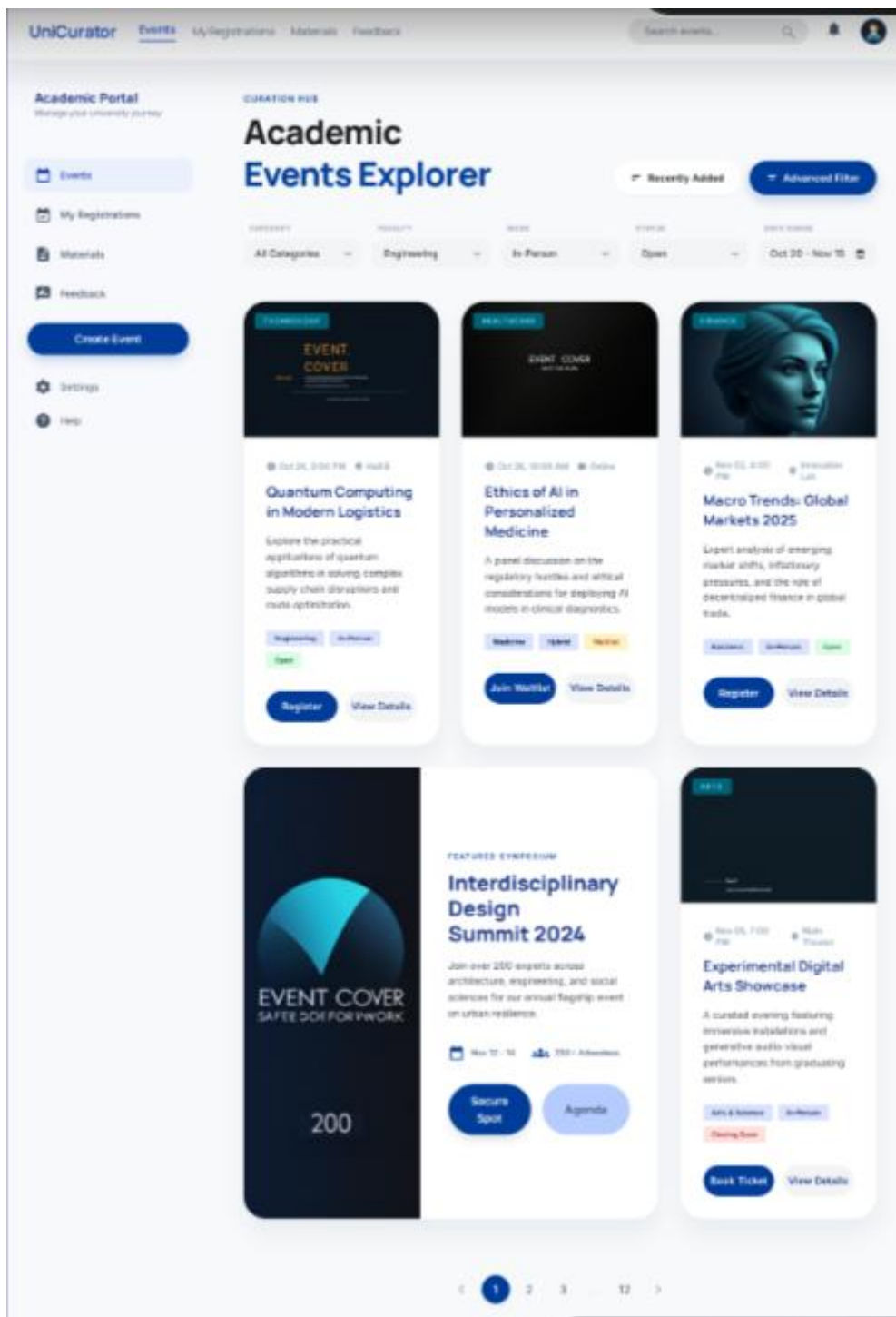


Figura 2. Designul interfeței SPA

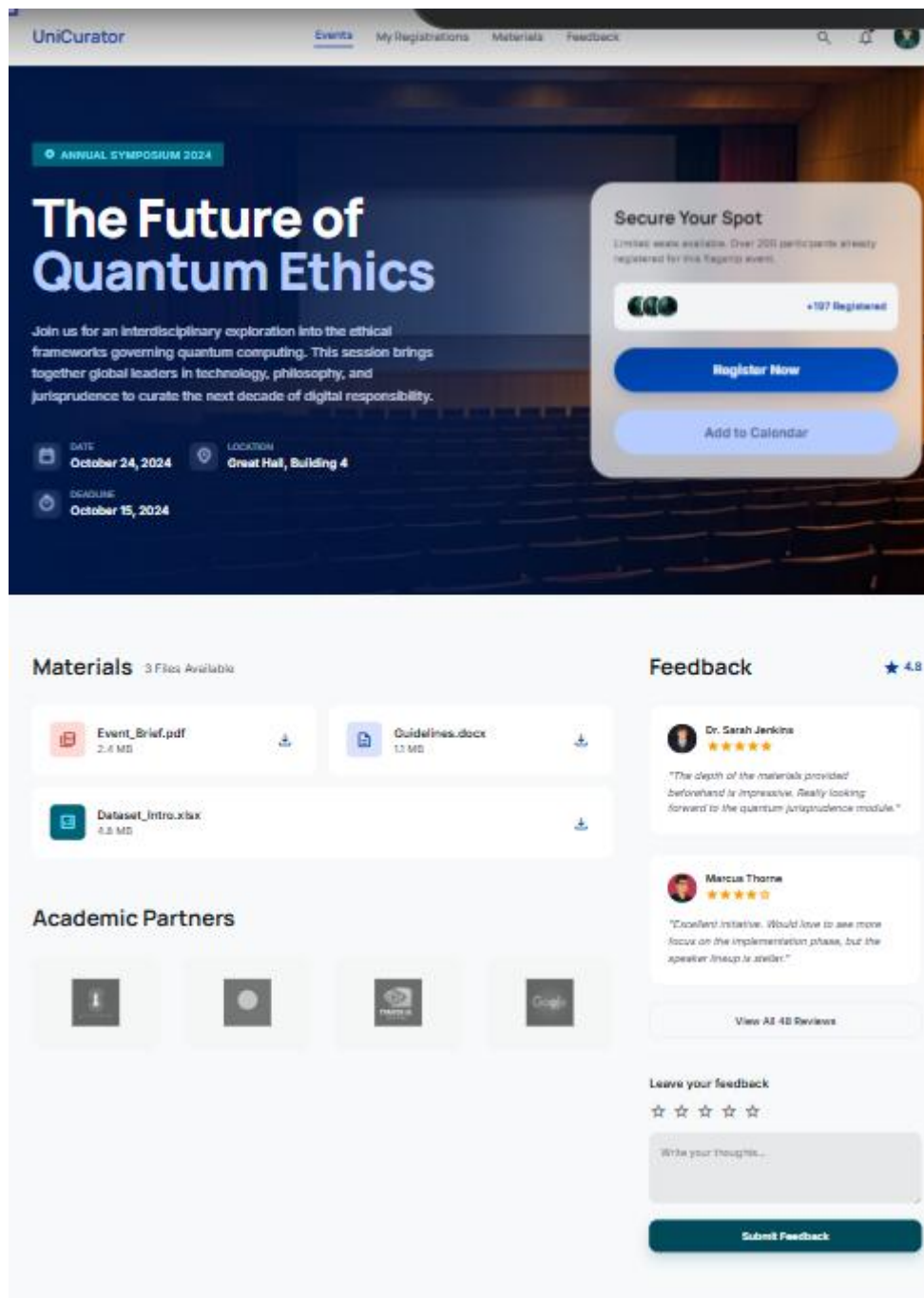


Figura 3. Designul interfeței SPA

Rezultatul generat a corespuns în mare măsură cerințelor definite în prompt. Mai mult, sistemul a adăugat elemente suplimentare de design care nu fuseseră specificate explicit:

- ierarhie vizuală mai clară și mai consistentă decât cea descrisă în prompt
- spațiere și aliniere optimizată a elementelor de interfață
- componente UI mai rafinate, cu tranziții și efecte hover de calitate profesională
- coerență vizuală globală între secțiuni, asigurând o experiență unitară a utilizatorului

Acest comportament ilustrează o caracteristică importantă a modelelor generative moderne: capacitatea de a interpreta contextul implicit al unei cereri și de a propune soluții care depășesc nivelul strict al instrucțiunilor oferite. Sistemul a acționat nu doar ca un executor al specificațiilor, ci ca un colaborator care aduce valoare adăugată în procesul de design.

Cu toate că rezultatul estetic a fost superior așteptărilor, au fost identificate și anumite limitări care au necesitat intervenție manuală ulterioară:

- Conținutul textual generat automat (titluri, descrieri, etichete) a utilizat date de tip placeholder care nu corespundeau contextului specific al aplicației
- Anumite interacțiuni dinamice erau schițate vizual, însă implementarea lor funcțională completă a necesitat cod JavaScript suplimentar
- Designul responsiv era optimizat prioritar pentru rezoluții desktop, necesitând ajustări pentru dispozitive mobile

Aceste limitări sunt tipice instrumentelor AI de prototipare rapidă și nu diminuează valoarea rezultatului ca punct de plecare solid pentru dezvoltarea ulterioară.

8.1 Debugging asistat de AI — un caz ilustrativ

O a doua etapă a utilizării AI în cadrul proiectului a vizat rezolvarea unei erori funcționale în logica de gestionare a sponsorilor. Problema constă în dublarea înregistrărilor în baza de date la fiecare operațiune de salvare, în loc de ștergerea entităților marcate pentru eliminare.

Prompt-ul inițial transmis asistentului AI a fost:

„În logica de update a unui eveniment, funcționalitatea actuală de ștergere a sponsorilor nu funcționează. Cum pot implementa corect ștergerea sponsorilor atunci când actualizez un eveniment?”

Răspunsul AI a propus următoarea logică:

- Sponsor existent (are id) + apăsare X → `sponsorsToDelete.push(id)`
- Sponsor nou (fără id) + apăsare X → dispăre doar din UI
- La Salvează: DELETE pentru fiecare id din `sponsorsToDelete`, POST pentru fiecare sponsor fără id

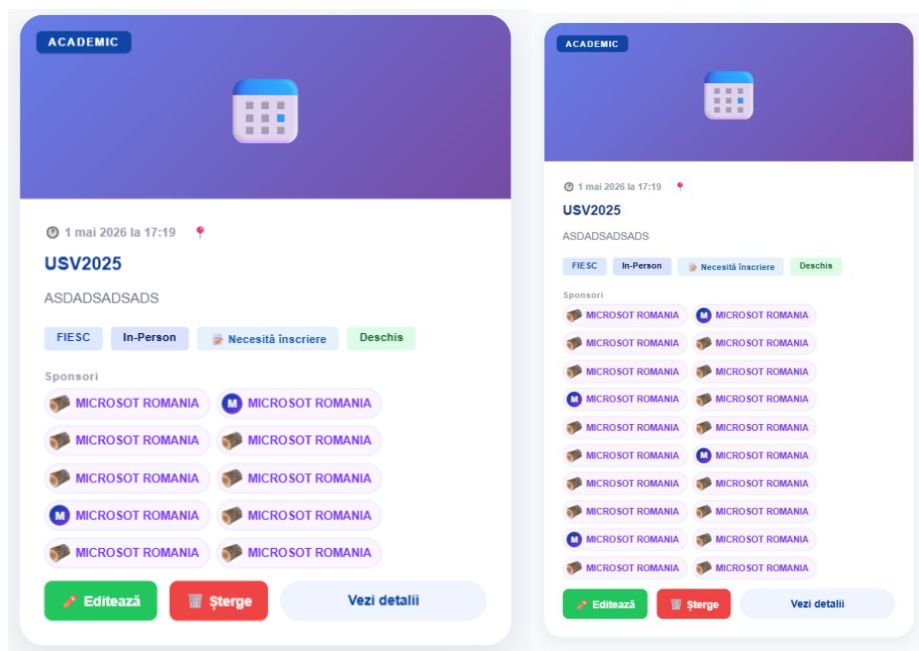


Fig 8.1 salvare sponsori

Deși logica părea corectă la nivel conceptual, în practică a apărut o problemă neașteptată: în loc să se ștergă, sponsorii erau dublați în baza de date la fiecare salvare. A fost necesară o a doua interacțiune cu AI, cu un prompt mai specific:

„Am o funcționalitate de update pentru un eveniment care are sponsori asociați. Problema este că:

- *sponsorii existenți nu sunt șterși atunci când nu mai sunt incluși în request*
- *la fiecare update (chiar fără modificări), sponsorii se dublează în baza de date*

Practic, doar intru și dau save, iar toți sponsorii existenți sunt inserați din nou.

Cum pot corecta logica astfel încât:

- *să nu mai apară duplicate*
- *să se facă update corect (sync) între lista trimisă și cea din baza de date?”*

Abia după 2-3 iterații succesive de prompt, în care au fost furnizate detalii suplimentare despre comportamentul efectiv al backend-ului și codul sursă, soluția corectă a fost identificată și implementată.

Experiența practică descrisă în acest capitol reflectă un tablou echilibrat al utilizării inteligenței artificiale în dezvoltarea software. Pe de o parte, instrumentele AI au accelerat semnificativ faza de prototipare a interfeței utilizator și au oferit un punct de plecare solid pentru procesul de depanare. Pe de altă parte, ambele cazuri au subliniat că rezultatele generate de AI necesită o validare atentă și iterații repetate. Calitatea finală a codului a fost direct proporțională cu specificitatea contextului furnizat și cu implicarea activă în verificarea și corectarea outputului AI.

10. Concluzii

AI a schimbat complet modul în care lucrează programatorii. Beneficiile sunt clare: dezvoltarea software este mai rapidă, sarcinile repetitive necesită mai puțin efort, documentația este mai bună, iar accesul la programare devine mai simplu. Aceste avantaje sunt susținute de studii serioase și rapoarte din industrie.

Totuși, există riscuri importante: calitatea codului poate varia, apar probleme de securitate și confidențialitate, dependența de AI poate deveni o problemă, iar aspectele legale și impactul asupra pieței muncii trebuie luate în considerare. Folosirea AI trebuie făcută responsabil, cu gândire critică și procese clare de verificare.

Concluzia principală este că AI nu înlocuiește programatorul, ci îl transformă. Viitorul aparține celor care știu să colaboreze bine cu AI, să verifice critic codul generat și să se concentreze pe aspectele care rămân profund umane în programare: nevoile utilizatorilor, deciziile arhitecturale și responsabilitatea pentru sistemele care afectează oameni reali.

Pe măsură ce AI devine mai puternică, granița dintre ajutor și autonomie se va schimba. Industria și societatea trebuie să se pregătească pentru această schimbare prin educație continuă, adaptarea învățământului și stabilirea unor reguli clare.

Bibliografie

- [1] Peng, S., Kalliamvakou, E., Croft, P., & Butter, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. NBER Working Paper No. 31085. National Bureau of Economic Research.
- [2] Dell'Acqua, F., McFowland, E., Mollick, E., et al. (2023). Navigating the jagged technological frontier: Field experimental evidence of the effects of AI on knowledge worker productivity and quality. Harvard Business School Working Paper 24-013.
- [3] GitHub. (2024). Octoverse 2024: The state of open source and rise of AI. GitHub Inc.
- [4] Stack Overflow. (2024). 2024 Developer Survey. Stack Overflow Insights.
- [5] McKinsey Global Institute. (2023). The economic potential of generative AI: The next productivity frontier. McKinsey & Company.
- [6] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. IEEE Symposium on Security and Privacy.
- [7] Liang, J. T., Yang, C., & Myers, B. A. (2023). A large-scale survey on the usability of AI programming assistants: Successes and challenges. ICSE 2024.
- [8] Gartner. (2024). Hype Cycle for Artificial Intelligence. Gartner Research.
- [9] Goldman Sachs. (2023). Generative AI: Too much spend, too little benefit? Goldman Sachs Global Investment Research.
- [10] Pluralsight. (2024). The state of upskilling 2024: AI's impact on tech skills. Pluralsight LLC.
- [11] Yang, J., Martins, P., Garg, T., et al. (2024). SWE-bench verified: A challenging benchmark for evaluating AI-based software engineers. arXiv preprint arXiv:2410.06992.
- [12] Free Software Foundation. (2021). FSF funded call for white papers on philosophical and legal questions around Copilot. FSF.org.
- [13] JetBrains. (2024). The state of developer ecosystem 2024. JetBrains s.r.o.
- [14] Andreessen Horowitz. (2024). AI coding tools market analysis. a16z Research Reports.

-
- [15] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- [16] Cui, K.Z., Demirer, M., Jaffe, S., Musolff, L., Peng, S. & Salz, T. (2024). *The Effects of Generative AI on High Skilled Work: Evidence from Three Field Experiments with Software Developers*. SSRN Working Paper No. 4945566. Available at: <https://ssrn.com/abstract=4945566>
- [17] Paradis, C. et al. (2024). *Google's internal evaluation of AI code completion tools*. Google Research — raport intern citat în Imas, A. (2026), What is the impact of AI on productivity? Substack.
- [18] Gambacorta, L., Qiu, H., Shan, S. & Rees, D. (2024). *AI and productivity at Ant Group (CodeFuse)*. Bank for International Settlements Working Paper. Citat în Imas, A. (2026).
- [19] Uplevel (2024). *The AI Productivity Paradox Research Report*. Faros.ai / Uplevel Blog. Available at: <https://www.faros.ai/blog/ai-software-engineering>
- [20] Perry, N. et al. (2023). *Do Users Write More Insecure Code with AI Assistants?* Proceedings of ACM CCS 2023. (Citat în PMC / systematic literature review, 2024.)
- [21] Adnan, M. et al. (2025). *Security vulnerabilities in AI-generated code samples — systematic analysis*. Citat în: AI Coding Assistants and the Rise of Bugs, Medium, 2025.
- [22]** Kalliamvakou, E. (2022). Research: quantifying GitHub Copilot's impact on developer productivity and happiness. GitHub Blog. Disponibil la: <https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/>
- [23]** Stack Overflow (2024). Developer Survey 2024 – AI Section. Disponibil la: <https://survey.stackoverflow.co/2024/ai>
- [24]** Stack Overflow (2025). Developer Survey 2025 – AI Section. Disponibil la: <https://survey.stackoverflow.co/2025/ai/>
- [25]** AMCIS 2024 Proceedings – The impact of GitHub Copilot on developer productivity from a software engineering body of knowledge perspective. AIS Electronic Library. Disponibil la: <https://aisel.aisnet.org/amcis2024>

[26] GitHub (2024). Octoverse 2024: The state of open source. Disponibil la:
<https://github.blog/news-insights/octoverse/octoverse-2024/>
